SAPIENZA
UNIVERSITÀ DI ROMA

Laboratorio di Ingegneria Informatica

Revisione di Python

Prof. Roberto **Navigli**

Python

Il linguaggio di programmazione

- **Linguaggio semplice e leggibile:** ha una sintassi chiara e intuitiva, simile al linguaggio naturale
- **Versatile e multiuso:** può essere utilizzato per sviluppo Web (Django, Flask), analisi dei dati (Pandas, NumPy), intelligenza artificiale e machine learning (TensorFlow, PyTorch), automazione, sviluppo di giochi (Pygame), ecc.
  - **Librerie e Framework Potenti:** Python offre un vasto ecosistema di librerie e framework che semplificano lo sviluppo e la risoluzione di problemi complessi in diversi settori
- **Comunità attiva:** Python ha una delle comunità più grandi e attive al mondo
  - Molte risorse, documentazione, tutorial e supporto disponibili
- **Portabilità e flessibilità:** Python è multipiattaforma e può essere eseguito su Windows, macOS, Linux e anche su dispositivi mobili
  - Inoltre è integrabile con altri linguaggi di programmazione come C, C++ e Java
- Nato nel 1991, 4 anni **prima di Java**

SAPIENZA UNIVERSITÀ DI ROMA

Sintassi semplice (ma tricky) (1)

- Niente parentesi graffe {} Python usa l'**indentazione** per definire i blocchi di codice (tipicamente utilizzando il **tab**), come i cicli e le funzioni

```
# Java
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

# Python
for i in range(5):
    print(i)
```

- **Attenzione:** tante volte non ci si rende conto del punto in cui finisce un blocco di codice se non si presta attenzione all'indentazione

SAPIENZA UNIVERSITÀ DI ROMA

Sintassi semplice (ma tricky) (2)

- Le istruzioni devono essere su una singola riga
- Altrimenti è necessario utilizzare il carattere speciale `\`

```python
# Calcolo dell'area totale di diverse forme geometriche
base_rettangolo = 10
altezza_rettangolo = 5
raggio_cerchio = 7

area_totale = (base_rettangolo * altezza_rettangolo) + \
              (3.14 * raggio_cerchio ** 2) + \
              (0.5 * base_rettangolo * altezza_rettangolo)

print(f"L'area totale è: {area_totale}")
```

- I commenti su una singola riga iniziano con `#`
- I commenti multiriga iniziano e terminano con `"""` (tre virgolette)

SAPIENZA UNIVERSITÀ DI ROMA

Convenzioni di case

- **snake_case:** le parole sono separate da un underscore (_) e tutte le lettere sono minuscole
  - Utilizzato per nomi di variabili, funzioni e metodi
  - Esempi: `nome_variabile`, `calcola_somma()`
- **PascalCase**: ogni parola inizia con una lettera maiuscola e non ci sono underscore
  - Utilizzato per i nomi delle classi, come in Java
  - Esempi: `NomeClasse`, `PersonaStudente`
- **UPPER_CASE**: tutte le lettere sono maiuscole, con le parole separate da underscore.
  - Utilizzato per le costanti
  - Esempi: `VALORE_COSTANTE`, `PI_GRECO = 3.14`

SAPIENZA UNIVERSITÀ DI ROMA

Controllo di flusso

- `if`, `elif`, `else` controllano le condizioni logiche:

```
if condizione:
    # blocco di codice
elif altra_condizione:
    # altro blocco di codice
else:
    # blocco di codice alternativo
```


SAPIENZA UNIVERSITÀ DI ROMA

Controllo di flusso

- `for` e `while` controllano le iterazioni:

```
for i in range(5):
    print(i)

while condizione:
    # blocco di codice
```

- `break` e `continue` controllano il flusso all'interno dei cicli:

```
for i in range(5):
    if i == 3:
        break  # interrompe il ciclo
    elif i == 1:
        continue  # salta alla prossima iterazione
    print(i)
```


SAPIENZA UNIVERSITÀ DI ROMA

Operatori logici e booleani

- Operatori logici:

  `&&` -> `and`, `||` -> `or`, `!` -> `not`

- Costanti booleane:

  `true` -> `True`, `false` -> `False`

```python
# Variabili booleane
is_raining = True
have_umbrella = False
is_weekend = True

# Uso di 'and': entrambe le condizioni devono essere vere
if is_raining and have_umbrella:
    print("Posso uscire senza bagnarmi.")
else:
    print("Meglio restare a casa.")  # Output: Meglio restare a casa.

# Uso di 'or': basta che una delle condizioni sia vera
if is_weekend or is_raining:
    print("Posso rilassarmi oggi!")  # Output: Posso rilassarmi oggi.

# Uso di 'not': inverte il valore booleano
if not have_umbrella:
    print("Devo prendere un ombrello.")  # Output: Devo prendere un ombrello.

# Combinazione di 'and', 'or' e 'not'
if is_raining and (have_umbrella or not is_weekend):
    print("Esco con l'ombrello.")
else:
    print("Resto a casa a guardare un film.")  # Output: Resto a casa a guardare un film.
```


SAPIENZA UNIVERSITÀ DI ROMA

Confronti e identità: differenza tra == e is

- `==` confronta il contenuto delle variabili o degli oggetti
- `is` confronta se le variabili puntano allo stesso oggetto in memoria

```python
# Confronto con numeri
a = 1000
b = 1000

print(a == b)  # Output: True (stesso valore)
print(a is b)  # Output: False (oggetti diversi in memoria)

# Confronto con stringhe
nome1 = "Alessandro"
nome2 = "Alessandro"

print(nome1 == nome2)  # Output: True (stesso valore)
print(nome1 is nome2)  # Output: True (stesso oggetto in memoria)
```

```python
# Due liste con lo stesso contenuto ma oggetti distinti
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]

print(lista1 == lista2)  # Output: True (stesso contenuto)
print(lista1 is lista2)  # Output: False (oggetti diversi in memoria)

# Assegnazione di riferimento alla stessa lista
lista3 = lista1
print(lista1 is lista3)  # Output: True (stesso oggetto in memoria)

# Modifica della lista e verifica
lista3.append(4)
print(lista1)  # Output: [1, 2, 3, 4] (modificata anche lista1)
```


SAPIENZA UNIVERSITÀ DI ROMA

Lettura e scrittura di file

Si crea un contesto e si crea un oggetto per la lettura del file (`_io.TextIOWrapper`):

```python
with open('file.txt', 'r') as file:
    contenuto = file.read()
```

```python
# Senza with: rischio di dimenticare di chiudere il file
file = open('file.txt', 'r')
contenuto = file.read()
file.close()  # Necessario per evitare memory leak

# Con with: chiusura automatica del file
with open('file.txt', 'r') as file:
    contenuto = file.read()
# Il file è automaticamente chiuso qui
```

e la scrittura di file:

```python
# Aprire o creare un file in modalità scrittura ('w')
with open('esempio.txt', 'w') as file:
    # Scrivere del testo nel file
    file.write("Ciao, questo è un file di testo creato con Python!\n")
    file.write("Aggiungiamo un'altra riga di testo.\n")
    file.write("Il file verrà chiuso automaticamente grazie al 'with'.\n")

print("Scrittura completata!")
```


SAPIENZA UNIVERSITÀ DI ROMA

Esempio codice di lettura file

```
with open('filename.txt', 'r', encoding='utf-8') as fr:
    for line in fr:
        print(line.strip())
```


SAPIENZA
UNIVERSITÀ DI ROMA

Tipizzazione dinamica

Al contrario di C/C++ e Java, in Python non si dichiara esplicitamente il tipo delle variabili

```
# Java
int numero = 5;

# Python
numero = 5  # Può diventare una stringa o qualsiasi altro tipo in seguito
numero = "Ciao"
```


SAPIENZA UNIVERSITÀ DI ROMA

Moduli, pacchetti e librerie (1)

- Un **modulo** non è altro che codice contenente funzioni, classi e variabili, **all'interno di un singolo file .py**
  - contiene codice, incluse funzioni, classi e variabili
- I moduli servono a organizzare e riutilizzare il codice **in modo più strutturato**

```python
# calcoli.py

def somma(a, b):
    return a + b

def moltiplicazione(a, b):
    return a * b
```

```python
# stringa.py

def unisci_stringhe(stringa1, stringa2):
    """Restituisce la concatenazione di due stringhe."""
    return stringa1 + stringa2

def ripeti_stringa(stringa, volte):
    """Restituisce la stringa ripetuta un certo numero di volte."""
    return stringa * volte
```


SAPIENZA UNIVERSITÀ DI ROMA

Moduli, pacchetti e librerie (2)

- Un **pacchetto** è una cartella contenente uno o più moduli e un file speciale chiamato `__init__.py`, che permette di trattare la cartella come un modulo importabile

```
mio_pacchetto/
    ├── __init__.py
    ├── modulo1.py
    └── modulo2.py
```

```
mio_pacchetto/
    ├── __init__.py
    ├── modulo1.py
    ├── modulo2.py
    └── sottopacchetto/
        ├── __init__.py
        └── modulo3.py
```


SAPIENZA UNIVERSITÀ DI ROMA

Moduli, pacchetti e librerie (2)

- Un **pacchetto** è una cartella contenente uno o più moduli e un file speciale chiamato `__init__.py`, che permette di trattare la cartella come un modulo importabile

```
mio_pacchetto/
    ├── __init__.py
    ├── modulo1.py
    └── modulo2.py
```

```
mio_pacchetto/
    ├── __init__.py
    ├── modulo1.py
    ├── modulo2.py
    └── sottopacchetto/
        ├── __init__.py
        └── modulo3.py
```

- Una **libreria** è una raccolta di moduli
  - E' costituita da un **insieme di file (.py) organizzati** in una **struttura di cartelle**
  - Spesso le librerie includono molte funzionalità raggruppate per uno scopo particolare
    - Es. la manipolazione dei dati (pandas), il machine learning (scikit-learn) o la creazione di interfacce grafiche (tkinter)
  - Le librerie Python spesso sono pacchetti

SAPIENZA UNIVERSITÀ DI ROMA

Importazione e installazione di moduli, pacchetti e librerie

Per importare moduli, pacchetti e librerie si utilizzano tre possibili sintassi:

- `import x` per mantenere chiaro il namespace del modulo (sarà necessario mantenere la sintassi `x.funzione()` o `x.Classe()`
- `from x import y` se hai bisogno solo di una o poche funzioni/classi `y` e vuoi evitare l'uso del namespace `x` (`y` potrà essere utilizzato senza dover accedere a `x.y`)
- `from x import *`, importa **tutto** ciò che si trova nel modulo `x`, ma è da sconsigliare: poco chiaro e incline agli errori

In Python si usa **pip** per installare librerie esterne:

```
pip install numpy
```


SAPIENZA UNIVERSITÀ DI ROMA

Stringhe e fstring

- Le stringhe in Python possono essere racchiuse da apici o virgolette (a scelta)
- E' possibile inserire incorporare variabili ed espressioni direttamente nelle stringhe con la sintassi {}:

```python
nome = "Michela"
età = 22

# f-string con variabili
messaggio = f"Ciao, mi chiamo {nome} e ho {età} anni."
print(messaggio)  # Output: Ciao, mi chiamo Michela e ho 22 anni.

# f-string con espressioni
print(f"Tra 5 anni, Michela avrà {età + 5} anni.")  # Output: Tra 5 anni, Michela avrà 27 ann

# f-string con formattazione numerica
prezzo = 49.99
print(f"Il prezzo è {prezzo:.2f} euro.")  # Output: Il prezzo è 49.99 euro.

# f-string con funzioni e metodi
print(f"Il nome in maiuscolo è {nome.upper()}.")  # Output: Il nome in maiuscolo è MICHELA.
```


SAPIENZA UNIVERSITÀ DI ROMA

Le funzioni str e int

“La mia età è “+str(eta)

SAPIENZA
UNIVERSITÀ DI ROMA

Stringhe e format

- Un metodo disponibile in precedenza era `str.format()`:

```
nome = "Michela"
età = 22

messaggio = "Ciao, mi chiamo {} e ho {} anni.".format(nome, età)
print(messaggio)  # Output: Ciao, mi chiamo Michela e ho 22 anni.

# Specificando gli indici
messaggio = "Mi chiamo {1} e ho {0} anni.".format(età, nome)
print(messaggio)  # Output: Mi chiamo Michela e ho 22 anni.

# Con i nomi dei parametri
messaggio = "Mi chiamo {nome} e ho {età} anni.".format(nome=nome, età=età)
print(messaggio)  # Output: Mi chiamo Michela e ho 22 anni.

# Formattazione numerica
print("Il prezzo è {:.2f} euro.".format(49.99))  # Output: Il prezzo è 49.99 euro.
```


SAPIENZA UNIVERSITÀ DI ROMA

Funzioni

- Le funzioni in Python sono molto più semplici e supportano funzionalità come i parametri di default:

```python
def saluta(nome="Mondo"):
    print(f"Ciao, {nome}!")

saluta("Python")  # Output: Ciao, Python!
saluta()  # Output: Ciao, Mondo!
```

- E gli argomenti passati per parole chiave:

```python
def descrivi_persona(nome, età, città):
    print(f"{nome} ha {età} anni e vive a {città}.")

# Chiamata della funzione con argomenti keyword:
descrivi_persona(nome="Michela", città="Roma", età=22)
```


SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dati avanzati

Python ha collezioni integrate come `list`, `tuple`, `set` e `dict` che sono molto potenti e facili da usare

	Mutable	Ordered	Indexing / Slicing	Duplicate Elements
List	✓	✓	✓	✓
Tuple	✗	✓	✓	✓
Set	✓	✗	✗	✗

SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dati avanzati

Liste

Le **liste** (classe `list`) sono **ordinate**, **modificabili** e **consentono elementi duplicati**:

```python
# Esempio di una lista con numeri e nomi
nomi = ["Luca", "Giorgia", "Marco", "Elena", "Luca"]
numeri = [10, 20, 30, 40, 50]

# Aggiungere elementi
nomi.append("Sara")

# Rimuovere elementi
numeri.remove(30)

# Accesso per indice
print(nomi[2])  # Output: Marco

# Modificare un elemento
nomi[1] = "Giulia"

# Risultato finale
print(nomi)   # Output: ['Luca', 'Giulia', 'Marco', 'Elena', 'Luca', 'Sara']
print(numeri) # Output: [10, 20, 40, 50]
```


SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dati avanzati

Tuple

Le **tuple** (classe `tuple`) sono simili alle liste ma sono **immutabili**, ovvero non è possibile modificarne il contenuto dopo la creazione

```
# Esempio di una tupla con numeri e nomi
dati_persona = ("Alessandro", 28, "Roma")
punteggi = (100, 95, 90, 85)

# Accesso per indice
print(dati_persona[0])  # Output: Alessandro

# Non è possibile modificare direttamente la tupla
# dati_persona[1] = 30  # Genererebbe un errore

# È possibile "sostituire" una tupla creando una nuova
dati_persona = ("Alessandro", 30, "Roma")
print(dati_persona)  # Output: ('Alessandro', 30, 'Roma')
```


SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dati avanzati

Insiemi

Gli **insiemi** (classe `set`) sono **non ordinati, non indicizzati** e **non consentono duplicati**:

```python
# Esempio di un set con numeri e nomi
nomi_unici = {"Francesca", "Giorgio", "Valeria", "Francesco", "Giorgio"}
numeri_unici = {1, 2, 3, 4, 5, 3, 2}

# Aggiungere elementi
nomi_unici.add("Matteo")

# Rimuovere elementi
numeri_unici.discard(3)

# Verifica presenza elemento
print("Valeria" in nomi_unici)  # Output: True

# Risultato finale
print(nomi_unici)   # Output: {'Francesco', 'Francesca', 'Valeria', 'Matteo', 'Giorgio'}
print(numeri_unici) # Output: {1, 2, 4, 5}
```


SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dati avanzati

Dizionari

I **dizionari** (classe `dict`) memorizzano **coppie chiave-valore**, sono **ordinati** (dalla versione 3.7 in poi) e **modificabili**

```python
# Esempio di un dizionario con numeri e nomi
studenti = {
    "ID1": "Matteo",
    "ID2": "Simona",
    "ID3": "Leonardo",
    "ID4": "Alice"
}

voti = {
    "Matematica": 28,
    "Inglese": 30,
    "Storia": 27
}

# Aggiungere una nuova coppia chiave-valore
studenti["ID5"] = "Martina"

# Modificare un valore esistente
voti["Inglese"] = 29

# Rimuovere una coppia chiave-valore
del studenti["ID2"]

# Accesso tramite chiave
print(studenti["ID1"])  # Output: Matteo

# Risultato finale
print(studenti)  # Output: {'ID1': 'Matteo', 'ID3': 'Leonardo', 'ID4': 'Alice', 'ID5': 'Mart
print(voti)      # Output: {'Matematica': 28, 'Inglese': 29, 'Storia': 27}
```


SAPIENZA UNIVERSITÀ DI ROMA

Tipi di dato avanzati

Creazione di oggetti vuoti

Per tutti e quattro i tipi (list, tuple, set, dict) si può usare il nome della classe con le parentesi aperte e chiuse per creare un oggetto vuoto di quel tipo.

SAPIENZA UNIVERSITÀ DI ROMA

Classi e oggetti

- In Java tutto è incentrato sul concetto di classe. In Python no.
- E' possibile (anche se **sconsigliabile**) creare un progetto senza OOP (Object-Oriented Programming)
- Sintassi per creare una classe e istanziare un oggetto:

```python
# Esempio di classe semplice in Python
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def parla(self):
        print(f"{self.nome} fa rumore")

cane = Animale("Cane")
cane.parla()
```

- **Nota bene:** in Python per istanziare un oggetto di una classe si usa una sintassi analoga a Java ma **SENZA new**

SAPIENZA UNIVERSITÀ DI ROMA

Classi

Self vs. this

- I metodi in Python sono funzioni dell'oggetto, quindi per essere applicate a un oggetto devono conoscere il "this", cioè il riferimento all'oggetto su cui devono essere invocate
- Questo "this" (richiamando la terminologia Java) in Python è `self`
- Ogni metodo di una classe che non sia statico deve prendere in input `self`

SAPIENZA UNIVERSITÀ DI ROMA

Classi

Metodi fondamentali: il costruttore __init__

- Il costruttore `__init__` viene invocato quando viene istanziato un oggetto con i parametri corrispondenti

```python
class Libro:
    def __init__(self, titolo, autore, pagine):
        self.titolo = titolo
        self.autore = autore
        self.pagine = pagine

# Creazione di un oggetto della classe Libro
libro = Libro("1984", "George Orwell", 328)

# Output utilizzando print()
print(libro)
```

- L'invocazione del costruttore avviene con la sintassi `NomeClasse(param`$_1$`, …, param`$_n$`)`
- I campi di una classe, così come le variabili esterne, non devono essere dichiarati

SAPIENZA UNIVERSITÀ DI ROMA

Classi

Metodi fondamentali: __str__

- Il metodo `__str__` viene ridefinito per ottenere una stampa personalizzata di un oggetto
  - Per una rappresentazione leggibile e informale dell'oggetto
  - Pensato principalmente per l'utente finale
  - Viene chiamato da `print()`, `str()` o altri contesti informali

```python
class Libro:
    def __init__(self, titolo, autore, pagine):
        self.titolo = titolo
        self.autore = autore
        self.pagine = pagine

# Creazione di un oggetto della classe Libro
libro = Libro("1984", "George Orwell", 328)

# Output utilizzando print()
print(libro)
```

```
<__main__.Libro object at 0x7f7d4e5d8d60>
```

```python
class Libro:
    def __init__(self, titolo, autore, pagine):
        self.titolo = titolo
        self.autore = autore
        self.pagine = pagine

    def __str__(self):
        return f"Libro: '{self.titolo}' di {self.autore}, {self.pagine} pagine"

# Creazione di un oggetto della classe Libro
libro = Libro("1984", "George Orwell", 328)

# Output utilizzando print()
print(libro)
```

```
Libro: '1984' di George Orwell, 328 pagine
```


SAPIENZA UNIVERSITÀ DI ROMA

Classi

Metodi di classe e metodi statici

In Python è possibile specificare metodi di classe e metodi statici:

- Un metodo di classe non prende self ma prende la classe `cls` in input ed è come un metodo statico in Java
- Un metodo statico invece non prende né `self` né `cls` in input, ma non può accedere o modificare lo stato della classe (né di un suo oggetto)

```python
class ContatoreOggetti:
    contatore = 0  # Attributo di classe

    def __init__(self):
        ContatoreOggetti.contatore += 1

    @classmethod
    def get_contatore(cls):
        return f"Numero di oggetti creati: {cls.contatore}"

# Creazione di istanze della classe
oggetto1 = ContatoreOggetti()
oggetto2 = ContatoreOggetti()

# Utilizzo del metodo di classe senza passare per un'istanza specifica
print(ContatoreOggetti.get_contatore())  # Output: Numero di oggetti creati: 2
```

```python
class Calcolatrice:

    @staticmethod
    def somma(a, b):
        return a + b

    @staticmethod
    def sottrai(a, b):
        return a - b

# Utilizzo del metodo statico senza creare un'istanza della classe
print(Calcolatrice.somma(10, 5))  # Output: 15
print(Calcolatrice.sottrai(10, 5))  # Output: 5

# È anche possibile chiamarlo su un'istanza, ma non è consigliato:
calcolatrice = Calcolatrice()
print(calcolatrice.somma(7, 3))  # Output: 10
```

Variabili locali, di classe, d'istanza

- **Variabile locale:** dichiarata all'interno di un metodo (o funzione) e accessibile solo dentro quel metodo/funzione
- **Variabile/attributo di classe:** definita all'interno della classe, ma fuori dai metodi, è condivisa da tutte le istanze della classe (= ne esiste una sola per tutti gli oggetti) ed è acceduta tramite il nome della classe (`Classe.attributo_di_classe`)
- **Variabile/attributo d'istanza:** definita all'interno del costruttore (`__init__`) e acceduta tramite `self`, ce n'è una per ogni istanza

```python
class Esempio:
    # Variabile di classe (condivisa da tutte le istanze della classe)
    variabile_classe = "Sono una variabile di classe"

    def __init__(self, valore):
        # Variabile d'istanza (specifica per ogni istanza della classe)
        self.variabile_istanza = valore

    def metodo(self):
        # Variabile locale (esiste solo all'interno di questo metodo)
        variabile_locale = "Sono una variabile locale"
        print(variabile_locale)  # Accesso alla variabile locale
        print(self.variabile_istanza)  # Accesso alla variabile d'istanza
        print(Esempio.variabile_classe)  # Accesso alla variabile di classe
```

```python
# Creazione di un oggetto della classe
oggetto = Esempio("Sono una variabile d'istanza")

# Chiamata al metodo
oggetto.metodo()

# Accesso alla variabile di classe dall'esterno della classe
print(Esempio.variabile_classe)

# Accesso alla variabile d'istanza dall'esterno della classe
print(oggetto.variabile_istanza)

# Tentativo di accesso alla variabile locale (genererà un errore perché esiste solo dentro il
# print(oggetto.variabile_locale)  # Questo darà un errore!
```


SAPIENZA UNIVERSITÀ DI ROMA

Classi

Campi protetti e privati

- In Python si chiamano attributi
- Con un “trick” di convenzione del case è possibile specificare campi protetti e privati:
- **Campo protetto** `_nome_attributo`: per convenzione, anteponendo un singolo underscore `_` si ottiene un campo protetto
  - anche se è comunque accessibile dall’esterno; è solo una convenzione
- **Campo privato** `__nome_attributo`: anteponendo un doppio underscore `__` si ottiene un campo privato (il campo viene rinominato in `_NomeClasse__nomecampo`)

SAPIENZA UNIVERSITÀ DI ROMA

Classi

Campi protetti e privati

```python
class Animale:
    def __init__(self, nome, specie):
        self._nome = nome  # Campo protetto
        self.specie = specie

    def _suono(self):
        return "Suono generico"

    def descrivi(self):
        return f"{self._nome} è un {self.specie} e fa '{self._suono()}'"

# Utilizzo
animale = Animale("Fido", "Cane")
print(animale.descrivi())  # Output: Fido è un Cane e fa 'Suono generico'

# Accesso diretto (non consigliato)
print(animale._nome)  # Output: Fido
```

```python
class Banca:
    def __init__(self, nome, saldo):
        self.nome = nome
        self.__saldo = saldo  # Campo privato

    def deposita(self, importo):
        if importo > 0:
            self.__saldo += importo

    def ottieni_saldo(self):
        return self.__saldo

# Utilizzo
conto = Banca("Mario", 1000)
conto.deposita(500)
print(conto.ottieni_saldo())  # Output: 1500

# Tentativo di accesso diretto
# print(conto.__saldo)  # Genera un errore: AttributeError

# Accesso tramite name mangling (non consigliato)
print(conto._Banca__saldo)  # Output: 1500
```


SAPIENZA UNIVERSITÀ DI ROMA

Ereditarietà

L'ereditarietà in Python è un concetto della programmazione orientata agli oggetti (OOP) che permette a una classe (detta **classe derivata** o **sottoclasse**) di **ereditare attributi** e **metodi** da un'altra classe (detta **classe base** o **superclasse**)

```python
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def fai_suono(self):
        print("L'animale fa un suono")

class Cane(Animale):
    def scodinzola(self):
        print(f"{self.nome} sta scodinzolando felice!")

# Creazione di un'istanza di Cane
fido = Cane("Fido")
print(fido.nome)  # Output: Fido
fido.fai_suono()  # Output: L'animale fa un suono (metodo ereditato da Animale)
fido.scodinzola() # Output: Fido sta scodinzolando felice! (nuovo metodo di Cane)
```


SAPIENZA UNIVERSITÀ DI ROMA

Ereditarietà

In Python è possibile ereditare da più classi:

```python
class Volante:
    def vola(self):
        print("Sto volando!")

class Pipistrello(Animale, Volante):
    def fai_suono(self):
        print("Squittio!")

bat = Pipistrello("Batman")
bat.fai_suono()  # Output: Squittio!
bat.vola()       # Output: Sto volando!
```


SAPIENZA UNIVERSITÀ DI ROMA

Il metodo super()

- `super()` è una funzione *built-in* che permette di accedere ai metodi e attributi della superclasse (la classe base) da una sottoclasse
- Spesso usata nel costruttore `__init__` per chiamare il costruttore della superclasse
  - Necessario per eseguire la parte di inizializzazione della superclasse
  - Al contrario di Java, `super().__init__()` **non viene chiamato automaticamente**

```python
class Animale:
    def __init__(self, nome):
        self.nome = nome
        print(f"Animale creato: {self.nome}")

class Cane(Animale):
    def __init__(self, nome, razza):
        # chiamata esplicita al costruttore della superclasse
        super().__init__(nome)
        self.razza = razza
        print(f"Cane creato: {self.nome}, Razza: {self.razza}")

# Esempio
fido = Cane("Fido", "Labrador")
```


SAPIENZA UNIVERSITÀ DI ROMA

Il metodo super()

Se non viene specificato il costruttore in una classe, verrà invocato quello della superclasse, ad es.:

```python
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def fai_suono(self):
        print("L'animale fa un suono")

class Cane(Animale):
    def scodinzola(self):
        print(f"{self.nome} sta scodinzolando felice!")

# Creazione di un'istanza di Cane
fido = Cane("Fido")
print(fido.nome)  # Output: Fido
fido.fai_suono()  # Output: L'animale fa un suono (metodo ereditato da Animale)
fido.scodinzola() # Output: Fido sta scodinzolando felice! (nuovo metodo di Cane)
```


SAPIENZA UNIVERSITÀ DI ROMA

Il metodo super()

E' possibile utilizzare `super()` anche per invocare l'implementazione di un metodo della superclasse:

```
class Animale:
    def parla(self):
        print("L'animale fa un verso")

class Cane(Animale):
    def parla(self):
        print("Il cane abbaia")
        super().parla()  # richiama la versione della superclasse

fido = Cane()
fido.parla()
```


SAPIENZA UNIVERSITÀ DI ROMA

Classi astratte

- In Python, le classi astratte vengono definite utilizzando il modulo `abc` (Abstract Base Class) e la corrispondente superclasse `ABC`
- Servono per definire un'interfaccia comune che deve essere implementata dalle sottoclassi
- Una classe astratta:
  - Non può essere istanziata direttamente.
  - Può contenere metodi concreti (implementati) e astratti (da implementare nelle sottoclassi)
  - Un metodo astratto viene definito con il decoratore `@abstractmethod`

SAPIENZA UNIVERSITÀ DI ROMA

Classi astratte

Esempio

```python
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimetro(self):
        pass
```

```python
class Quadrato(Forma):
    def __init__(self, lato):
        self.lato = lato

    def area(self):
        return self.lato ** 2

    def perimetro(self):
        return 4 * self.lato

class Cerchio(Forma):
    def __init__(self, raggio):
        self.raggio = raggio

    def area(self):
        return math.pi * self.raggio ** 2

    def perimetro(self):
        return 2 * math.pi * self.raggio

# Esempio di utilizzo:
q = Quadrato(5)
c = Cerchio(3)

print(f"Quadrato - Area: {q.area()}, Perimetro: {q.perimetro()}")
print(f"Cerchio - Area: {c.area()}, Perimetro: {c.perimetro()}")
```

Overriding

- Si ha l'**overriding** quando una sottoclasse ridefinisce un metodo della superclasse
- Il metodo della sottoclasse sostituisce quello della classe genitore quando viene chiamato su un'istanza della sottoclasse
- Il metodo nella sottoclasse ha lo **stesso nome** e la **stessa signature** (**parametri**) di quello nella superclasse

```python
class Animale:
    def suono(self):
        return "Alcuni animali fanno rumore"

class Cane(Animale):
    def suono(self):  # Overriding del metodo della classe padre
        return "Bau Bau"

class Gatto(Animale):
    def suono(self):  # Overriding del metodo della classe padre
        return "Miao Miao"

# Creazione delle istanze
animale_generico = Animale()
cane = Cane()
gatto = Gatto()

# Output
print(animale_generico.suono())  # "Alcuni animali fanno rumore"
print(cane.suono())  # "Bau Bau"
print(gatto.suono())  # "Miao Miao"
```


SAPIENZA UNIVERSITÀ DI ROMA

Polimorfismo

Con l'overriding

- Un concetto fondamentale della programmazione orientata agli oggetti (OOP)
- Permette di usare un'unica interfaccia per diversi tipi di dati
- Lo stesso metodo **ridefinito tramite overriding** può avere comportamenti diversi a seconda dell'oggetto su cui viene chiamato

```python
class Animale:
    def suono(self):
        return "Alcuni animali emettono suoni"

class Cane(Animale):
    def suono(self):  # Override del metodo suono()
        return "Bau Bau"

class Gatto(Animale):
    def suono(self):  # Override del metodo suono()
        return "Miao Miao"

# Esempio di polimorfismo
def fai_suonare(animale):
    print(animale.suono())

# Creiamo istanze di classi diverse
animali = [Cane(), Gatto(), Animale()]

for animale in animali:
    fai_suonare(animale)  # Output dipende dalla classe dell'oggetto
```

```
Bau Bau
Miao Miao
Alcuni animali emettono suoni
```


SAPIENZA
UNIVERSITÀ DI ROMA

Polimorfismo

Con le funzioni

Possiamo scrivere funzioni che accettano oggetti diversi e usano metodi comuni

```python
class Rettangolo:
    def __init__(self, larghezza, altezza):
        self.larghezza = larghezza
        self.altezza = altezza

    def area(self):
        return self.larghezza * self.altezza

class Cerchio:
    def __init__(self, raggio):
        self.raggio = raggio

    def area(self):
        return 3.14 * self.raggio * self.raggio
```

```python
# Funzione polimorfica
def stampa_area(figura):
    print(f"L'area è: {figura.area()}")

# Oggetti di classi diverse con un metodo comune
rettangolo = Rettangolo(4, 5)
cerchio = Cerchio(3)

stampa_area(rettangolo)  # Output: L'area è: 20
stampa_area(cerchio)     # Output: L'area è: 28.26
```


SAPIENZA UNIVERSITÀ DI ROMA

Polimorfismo

Tramite overloading dell'operatore

Modifichiamo il comportamento dell'operatore somma (`__add__`) sovraccaricandolo nel caso della somma di due punti:

```python
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, altro_punto):
        return Punto(self.x + altro_punto.x, self.y + altro_punto.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Punto(1, 2)
p2 = Punto(3, 4)

p3 = p1 + p2  # Overloading dell'operatore +
print(p3)  # Output: (4, 6)
```


SAPIENZA UNIVERSITÀ DI ROMA

Overloading

- Si ha l'**overloading** quando si definiscono più versioni dello stesso metodo con firme diverse
- Python non supporta il vero overloading come in Java o C++, ma può essere simulato usando argomenti predefiniti o `*args` e `**kwargs` (**keyword arguments**)

```python
class Matematica:
    def somma(self, a, b, c=0):  # Overloading tramite argomento opzionale
        return a + b + c

# Creazione dell'oggetto
calcoli = Matematica()

# Chiamata con due argomenti
print(calcoli.somma(2, 3))  # Output: 5

# Chiamata con tre argomenti
print(calcoli.somma(2, 3, 4))  # Output: 9
```


SAPIENZA UNIVERSITÀ DI ROMA

Overloading

- Si ha l'**overloading** quando si definiscono più versioni dello stesso metodo con firme diverse
- Python non supporta il vero overloading come in Java o C++, ma può essere simulato usando argomenti predefiniti o `*args` e `**kwargs` (**keyword arguments**)
  - `*args` raccoglie gli argomenti in una tupla
  - `**kwargs` li raccoglie in una dizionario

```python
class Matematica:
    def somma(self, *numeri):  # Overloading simulato con *args
        return sum(numeri)

# Creazione dell'oggetto
calcoli = Matematica()

# Chiamate con un numero variabile di argomenti
print(calcoli.somma(2, 3))       # Output: 5
print(calcoli.somma(2, 3, 4))    # Output: 9
print(calcoli.somma(1, 2, 3, 4)) # Output: 10
```

```python
class Persona:
    def __init__(self, **kwargs):
        self.nome = kwargs.get("nome", "Sconosciuto")
        self.età = kwargs.get("età", 0)
        self.città = kwargs.get("città", "Non specificato")

    def mostra_info(self):
        print(f"Nome: {self.nome}, Età: {self.età}, Città: {self.città}")

p1 = Persona(nome="Michela", età=22, città="Milano")
p2 = Persona(nome="Giovanni")

p1.mostra_info()  # Output: Nome: Michela, Età: 22, Città: Milano
p2.mostra_info()  # Output: Nome: Giovanni, Età: 0, Città: Non specificato
```


SAPIENZA UNIVERSITÀ DI ROMA

Eccezioni

- La gestione delle eccezioni è simile a Java, ma con una sintassi differente:

```python
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print(f"Errore: {e}")
finally:
    print("Blocco finally eseguito")
```

- `raise` lancia un'eccezione:

```python
def controlla_eta(eta):
    if eta < 18:
        raise ValueError("L'età deve essere almeno 18 anni")
```


SAPIENZA UNIVERSITÀ DI ROMA

Programmazione funzionale

- Le funzioni lambda permettono di definire **funzioni anonime**:

```python
# Definizione della funzione lambda
quadrato = lambda x: x ** 2

# Uso della funzione
risultato = quadrato(5)
print(risultato)  # Output: 25
```

```python
# Lista di tuple (nome, età)
persone = [("Luca", 30), ("Anna", 22), ("Marco", 25)]

# Ordinare per età utilizzando una funzione lambda
persone_ordinate = sorted(persone, key=lambda persona: persona[1])

print(persone_ordinate)
# Output: [('Anna', 22), ('Marco', 25), ('Luca', 30)]
```

- È possibile anche il passaggio di funzioni come parametri:

```python
# Lista di tuple (nome, età)
persone = [("Luca", 30), ("Anna", 22), ("Marco", 25)]

# Definisci una funzione per estrarre l'età dalla tupla
def estrai_eta(persona):
    return persona[1]

# Ordina la lista di persone usando la funzione personalizzata
persone_ordinate = sorted(persone, key=estrai_eta)

print(persone_ordinate)
# Output: [('Anna', 22), ('Marco', 25), ('Luca', 30)]
```


SAPIENZA UNIVERSITÀ DI ROMA

Funzioni di grande utilità

- `map(funzione, iterabile)`: applica una funzione a ogni elemento di un iterabile (lista, tupla, ecc.) e restituisce un oggetto mappato (che può essere convertito in una lista, tupla, ecc.):

```
numeri = [1, 2, 3, 4, 5]
raddoppiati = list(map(lambda x: x * 2, numeri))
print(raddoppiati)  # Output: [2, 4, 6, 8, 10]
```


SAPIENZA UNIVERSITÀ DI ROMA

Funzioni di grande utilità

- `map(funzione, iterabile)`: applica una funzione a ogni elemento di un iterabile (lista, tupla, ecc.) e restituisce un oggetto mappato (che può essere convertito in una lista, tupla, ecc.):

```
numeri = [1, 2, 3, 4, 5]
raddoppiati = list(map(lambda x: x * 2, numeri))
print(raddoppiati)  # Output: [2, 4, 6, 8, 10]
```

- `filter`: applica una funzione a ogni elemento di un iterabile e restituisce solo gli elementi per i quali la funzione restituisce `True`

```
numeri = [1, 2, 3, 4, 5]
numeri_pari = list(filter(lambda x: x % 2 == 0, numeri))
print(numeri_pari)  # Output: [2, 4]
```


SAPIENZA UNIVERSITÀ DI ROMA

Funzioni di grande utilità

- map(funzione, iterabile): applica una funzione a ogni elemento di un iterabile (lista, tupla, ecc.) e restituisce un oggetto mappato (che può essere convertito in una lista, tupla, ecc.):

```
numeri = [1, 2, 3, 4, 5]
raddoppiati = list(map(lambda x: x * 2, numeri))
print(raddoppiati)  # Output: [2, 4, 6, 8, 10]
```

- filter: applica una funzione a ogni elemento di un iterabile e restituisce solo gli elementi per i quali la funzione restituisce True

```
numeri = [1, 2, 3, 4, 5]
numeri_pari = list(filter(lambda x: x % 2 == 0, numeri))
print(numeri_pari)  # Output: [2, 4]
```

- sorted(iterabile, key=None, reverse=False): restituisce una nuova lista ordinata dagli elementi di un iterabile
  - Può ordinare numeri o stringhe e supporta l'ordinamento personalizzato tramite il parametro key

```
parole = ["mela", "banana", "kiwi", "ananas"]
parole_ordinate = sorted(parole, key=len)
print(parole_ordinate)  # Output: ['kiwi', 'mela', 'banana', 'ananas']
```


SAPIENZA UNIVERSITÀ DI ROMA

Non è necessario il main

- Un programma Python può essere scritto anche con una sola riga di codice utilizzando il paradigma imperativo e **senza main**
- E' però possibile specificare del codice da eseguire **solo** se il modulo è stato lanciato direttamente:

```
# In Java:
public class Main {
    public static void main(String[] args) {
        System.out.println("Ciao!");
    }
}

# In Python:
if __name__ == "__main__":
    print("Ciao!")
```

- Altrimenti il codice viene sempre eseguito quando si importa il modulo da un altro modulo

SAPIENZA UNIVERSITÀ DI ROMA

Type hinting

- Introdotto in Python 3.5 (con il modulo `typing`) che permette di specificare i tipi di variabili, parametri e valori di ritorno delle funzioni
- **Non è obbligatorio** e **non influisce** direttamente **sull'esecuzione del codice** (Python rimane un linguaggio dinamico), ma aiuta a **migliorare la leggibilità**, il **debugging** e la **qualità del codice**
  - soprattutto in progetti strutturati

```python
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

- `nome_parametro: str` indica che il parametro name deve essere una stringa
- `-> str` indica che la funzione restituirà una stringa

SAPIENZA UNIVERSITÀ DI ROMA

Type hinting

Variabili e tipi complessi

- E' possibile specificare il type hinting anche per le variabili:

```python
age: int = 25
name: str = "Mario"
is_student: bool = True
```

- Il modulo `typing` fornisce strumenti per tipi più complessi:

```python
from typing import List, Dict, Tuple, Optional, Union

# Lista di interi
numbers: List[int] = [1, 2, 3]

# Dizionario con chiavi stringa e valori float
scores: Dict[str, float] = {'math': 95.0, 'science': 89.5}

# Tupla con un intero e una stringa
person: Tuple[int, str] = (25, "Mario")

# Valore opzionale (può essere un intero o None)
age: Optional[int] = None

# Unione di tipi (può essere int o str)
identifier: Union[int, str] = "123ABC"
```

```python
from typing import Callable

# Funzione che accetta una funzione come parametro
def process_data(data: List[int], processor: Callable[[int], str]) -> List[str]:
    return [processor(item) for item in data]
```


SAPIENZA
UNIVERSITÀ DI ROMA

Type hinting

Vantaggi e svantaggi

- Aiuto da parte degli editor (autocomplete e suggerimenti)
  - Ad esempio, in VSCode, installate l'estensione Pylance
  - Oppure è possibile ottenere un'analisi approfondita con la libreria `mypy`
- Codice documentato meglio e meno soggetto ad errore
- E' un po' un arteficio: non viene verificato a runtime!!!

SAPIENZA UNIVERSITÀ DI ROMA

Metodi e librerie per esercizi

Esercizi: librerie e funzioni utili

- **Benchmarking:** `time.perf_counter()` per una misurazione precisa del tempo
- **Grafici:** `matplotlib` è una libreria integrata a Python per fare semplici grafici
- **Docstring:** le docstring sono il metodo standard di documentare le funzioni e le classi in Python

SAPIENZA UNIVERSITÀ DI ROMA

Misurazione precisa del tempo: time.perf_counter

Esercizi: librerie e funzioni utili 1

- **Scopo:** `time.perf_counter()` è una funzione nel modulo `time` (parte della libreria standard di Python) specificamente progettata per il benchmarking
  - Fornisce un contatore delle prestazioni ad alta risoluzione
- **Come funziona:** restituisce un valore float che rappresenta il tempo in secondi (incluse parti frazionarie come millisecondi e nanosecondi)
  - Il valore assoluto restituito non è significativo; solo la differenza tra due chiamate è rilevante
- **Caso d'uso ideale:** perfetto per misurare durate brevi e confrontare il tempo di esecuzione di diversi segmenti di codice

```python
import time

start_time = time.perf_counter()
# ... codice da misurare ...
end_time = time.perf_counter()
tempo_trascorso = end_time - start_time
print(f"Tempo trascorso: {tempo_trascorso:.4f} secondi")
```


SAPIENZA UNIVERSITÀ DI ROMA

Creazione di visualizzazioni con Matplotlib

Esercizi: librerie e funzioni utili 2

- Matplotlib è la libreria Python più popolare e fondamentale per la creazione di visualizzazioni statiche, animate e interattive
- **Versatile:** supporta un'ampia gamma di tipi di grafici (grafici 2D, istogrammi, grafici 3D e altro).
- **Personalizzabile:** offre un ampio controllo sull'aspetto del grafico (colori, etichette, titoli, legende, assi, ecc.).

```python
import matplotlib.pyplot as plt

# Dati di esempio
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 3, 5]

# Crea un semplice grafico a linee
plt.plot(x, y)

# Aggiungi etichette e titolo
plt.xlabel("Asse X")
plt.ylabel("Asse Y")
plt.title("Semplice Grafico a Linee")

# Mostra il grafico
plt.show()
```


SAPIENZA UNIVERSITÀ DI ROMA

Documentare il codice con le docstring

Esercizi: librerie e funzioni utili 3

- Le docstring (stringhe di documentazione) sono stringhe letterali utilizzate per documentare moduli, funzioni, classi e metodi Python
  - Sono il modo standard per documentare il codice Python

- **Posizionamento:** una docstring deve essere la prima istruzione all'interno dell'oggetto che documenta (ad esempio, la prima riga all'interno di una definizione di funzione)

- **Sintassi:** racchiuse tra triple virgolette o apici (""""Docstring va qui"""" o '''Docstring va qui'''). Le triple virgolette sono la convenzione standard.

- **Scopo:**
  - Descrivere cosa fa il codice (non come lo fa)
  - Fornire informazioni per gli utenti del codice (incluso te stesso in futuro!)

SAPIENZA UNIVERSITÀ DI ROMA

Documentare il codice con le docstring (esempio)

Esercizi: librerie e funzioni utili 3.1

```python
def mia_funzione(param1, param2):
    """
    Questa è una riga di riepilogo che descrive lo scopo della funzione.

    Questa è una descrizione più dettagliata, che può estendersi su più righe.
    Spiega i parametri e il valore di ritorno.

    Args:
        param1 (int): Descrizione di param1.
        param2 (str): Descrizione di param2.

    Returns:
        bool: True se ha successo, False altrimenti.
    """
    # Implementazione della funzione...
    return True
```


SAPIENZA UNIVERSITÀ DI ROMA

Esercizi

Esercizi: raccomandazioni

- Svolgere prima tutti gli esercizi e poi alla fine svolgere i bonus.
- Ricordate di svolgere ogni esercizio che segue come se fosse parte di un grande progetto industriale con varie altre persone nel team, quindi:
  - Seguire la convezione di casing standard
  - Usare i type hints
  - Usare le docstring
  - Organizzare il codice usando delle classi

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 1: Simulazione E-store

Ripasso delle basi: pacchetti, sottopacchetti e moduli.

- Crea un pacchetto e_store con i sottopacchetti:
  - customer che contiene GenericCostumer, NormalCostumer, PromotionalCostumer (PromotionalCostumer che ha 5%di sconto)
  - store_item, che contiene GenericItem, NormalItem, ForeignItem (che deve essere gestito in modo speciale dato che viene da fuori la nazione)
  - store_inventory che contiene StoreInventory che salva gli oggetti con la loro quantità
  - store che contiene Store con store_inventory e il il denaro del negozio.
  - ui che contiene la logica per cercare prodotti, comprare dai fornitori, vendere at clienti etc... (da terminale)
- Scrivi una funzione main che importa la UI e permette di simulare la attività di base e di stampare in modo leggibile tutte le componenti.
- (La funzionalità è basilare, l'esercizio è per sviluppare l'organizzazione in pacchetti, sottopacchetti e moduli).

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 1: Simulazione E-store (2)

Ripasso delle basi: pacchetti, sottopacchetti e moduli.

Funzionalità:

- L'inventario ha degli oggetti con una certa quantità e un certo prezzo
- Il cliente ha una certa quantità di denaro e una password
- Nella UI (da terminale), gli oggetti vengono stampati con quantità e prezzo, viene richiesto all'utente cosa vuole comprare e se ci sono a abbastanza oggetti e il cliente ha abbastanza soldi e la password corretta allora l'acquisto può avvenire. Poi l'inventario viene stampato nuovamente e tutto si ripete fino a che l'utente scrive "quit".
- Se l'oggetto viene dall'estero (ForeignItem) aumentare il prezzo del 20%

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 1: Simulazione E-store (3)

Ripasso delle basi: pacchetti, sottopacchetti e moduli.

- Come gestire gli import relativi in Python
- Organizzare le cartelle del progetto come segue (con anche gli altri pacchetti all'interno, allo stesso livello di `inventory` e `item`), la cartella `src` è uno standard (il comando tree mostra semplicemente la struttura delle cartelle):

```
root@2408eb2c18b6:/mnt/data/riccardo/laboratorio_di_programmazione/lezioni/lezione_2# tree
.
└── e_store_project
    └── src
        └── e_store
            ├── inventory
            │   └── inventory.py
            └── item
                └── item.py

5 directories, 2 files
```


SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 1: Simulazione E-store (4)

Ripasso delle basi: pacchetti, sottopacchetti e moduli.

- Python deve sapere dove cercare il pacchetto padre `e_store` per questo è necessario usare la variabile d'ambiente `PYTHONPATH`
- Questo è un esempio di file `inventory.py` che importa `item.py` (esempio semplificato). `item.py` semplicemente definisce la classe `Item` che si può inizializzare con un certo prezzo.

```
from e_store.item.item import Item

x = Item(price=15)

print(x)
```


SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 1: Simulazione E-store (5)

Ripasso delle basi: pacchetti, sottopacchetti e moduli.

- Se la `PYTHONPATH` non è impostata Python non riesce a trovare il pacchetto `e_store` e fallisce, se invece è impostata con `export` da terminale se eseguite da terminale o nelle impostazioni di VSCode se eseguite da UI di VSCode (il triangolino), lo trova e il codice si esegue correttamente (la classe `Item` non ha un metodo `__repr__`, quindi questo output è corretto). La `PYTHONPATH` deve essere impostata al path assoluto della vostra cartella `src`, per esempio nel mio computer:

```
root@2408eb2c18b6:/mnt/data/riccardo/laboratorio_di_programmazione/lezioni/lezione_2# python '/mnt/data/riccardo/laboratorio_di_progra
mmazione/lezioni/lezione_2/e_store_project/src/e_store/inventory/inventory.py'
Traceback (most recent call last):
  File "/mnt/data/riccardo/laboratorio_di_programmazione/lezioni/lezione_2/e_store_project/src/e_store/inventory/inventory.py", line 1
, in <module>
    from e_store.item.item import Item
ModuleNotFoundError: No module named 'e_store'
root@2408eb2c18b6:/mnt/data/riccardo/laboratorio_di_programmazione/lezioni/lezione_2# export PYTHONPATH="/mnt/data/riccardo/laboratori
o_di_programmazione/lezioni/lezione_2/e_store_project/src"
root@2408eb2c18b6:/mnt/data/riccardo/laboratorio_di_programmazione/lezioni/lezione_2# python '/mnt/data/riccardo/laboratorio_di_progra
mmazione/lezioni/lezione_2/e_store_project/src/e_store/inventory/inventory.py'
<e_store.item.item.Item object at 0x7f42aed990a0>
```

Esercizio 2: Indovina il numero

Ripasso delle basi: variabili, loop, condizionali, file, eccezioni

- Il computer sceglie un numero da 0 a N, e un giocatore deve provare a indovinarlo. Se il numero è troppo alto il giocatore riceve l'output "troppo alto", se è troppo basso "troppo basso" e se è giusto il gioco finisce, dopo "nome del giocatore, numero da indovinare, numero di tentativi, tentativi, data, ora" vengono salvati su un file tsv. "tentativi" sono tutti i tentativi del giocatore. Usa try / except per gestire il caso in cui il giocatore da un input non numerico
- Il codice deve supportare SENZA MODIFICHE una classe HumanPlayer che è un giocatore con input/output da terminale e un CpuPlayer che gioca in automatico (ex: old_guess+1 se aveva scelto basso o old_guess-1 se aveva scelto alto)
- HumanPlayer e CpuPlayer devono essere classi concrete di Player (classe astratta)

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 2 BONUS: Indovina il numero *FAST*

Ripasso delle basi: variabili, loop, condizionali, file

- Bonus: è sempre possibile vincere a questo gioco con numeri da 0 a 1000 miliardi in 40 mosse o meno. Implementare un FastCpuPlayer con questa caratteristica e testarlo in 1000 giochi salvando tutto su file come nella slide precedente.

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 3: Sentiment analysis “old school”

Ripasso delle basi: file, strutture dati

- L’input è un file tsv con colonne, “text”, “label” dove “text” è una recensione (in inglese) e “label” indica se le emozioni espresse sono positive o negative.
- Leggi le recensioni dal file, per ognuna conta le “parole felici” (happy, good, etc…) e le “parole tristi” (sad, bad, etc…), in base a questo determina se la recensione è positiva o negativa, e calcola l’accuracy (che percentuale di volte le previsioni sono corrette), confrontando con il “basic model” che conta la percentuale di recensioni positive e negative e risponde sempre con la classe di maggioranza.
- Anche quì WordModel e BasicModel derivano da una classe astratta Model
- Bisogna usare solo la metà delle recensioni per decidere quali parole usare e la classe di maggioranza, e poi di valutare la performance sull’altra metà.
- Scrivere un nuovo file tsv “text”, “label”, “predicted label”

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 4: Tris

Ripasso delle basi: Classi astratte e modularità

- Scrivere un programma per giocare a Tris.
- Deve essere possibile supportare sia giocatori umani che Cpu (anche casuale)
- Deve essere possibile giocare sia (da terminale) sia con ASCII che con UNICODE
- Devono essere possibili tutte le combinazioni giocatore-giocatore-UI (umano-umano-ui_unicode, umano-cpu-ui_unicode, umano-cpu-ui_ascii, cpu-cpu-ui_ascii)
- La logica del tris è molto semplice, quindi l'obiettivo principale di questo esercizio è assicurarsi di costruire un sistema con più di un elemento modulare, usando l'interfaccia della classe astratta per assicurarsi che tutto funzioni.
- Mini-bonus: è possibile giocare a tris perfettamente seguendo poche semplici regole, implementare un'altra CpuPlayerGood che gioca perfettamente

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 5: Benchmark della libraria standard

Ripasso delle basi: liste, set, benchmark

- I set e i dizionari in Python sono importanti perché permettono di verificare se un item appartiene alla collezione con complessità temporale O(1)
- Fare un benchmark con liste e set casuali di varie dimensioni, misurare il tempo necessario per controllare se un item è nella lista/set per diverse dimensioni della lista set. Ripetere il test per ogni dimensioni 10 volte per ridurre la variazione casuale.
- Scrivere i risultati su file e fare un grafico tempo vs numero di item per lista, set e dizionario. Usare matplotlib, perf_counter.
- (mini-bonus: il set sarà migliaia di volte più veloce della lista o anche di più in caso contengano molti item, come fare un grafico leggibile?)

SAPIENZA UNIVERSITÀ DI ROMA

Esercizio 6: Debugging

Ripasso delle basi: Un po' di tutto

- Ho scritto alcuni semplici programmi che contengono dei bug, ovvero o sempre o solo in determinate condizioni non funzionano correttamente.
- Questi bug rappresentano errori che possono succedere comunemente sia nella programmazione in generale che in Python in particolare.
- Ogni file contiene almeno un bug (ma potrebbe contenerne di più)
- Utilizzare il debugger integrato in VSCode per trovare e risolvere i problemi in questi programmi.

SAPIENZA UNIVERSITÀ DI ROMA